

Lecture 4.

Inside Python: Objects, References & Runtime.

Software Development & Python

Ivan Solomatin

MIPT

From language to CPU

Where are we now?



Today: what does the Python runtime actually manipulate?

Not machine integers and “variables in boxes” — mostly objects and references.

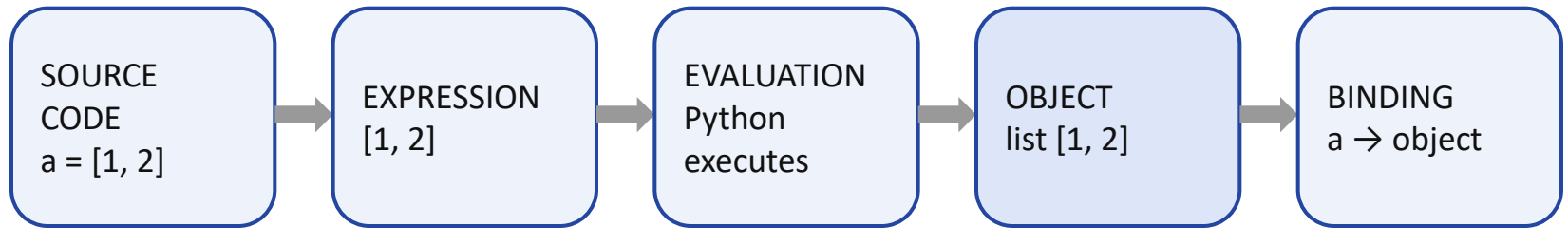
Outline

- Why objects?
- Objects & Names
- Mutation & Identity
- Calls & Containers
- Copying
- Lifetime & Garbage Collector

Why object abstraction?

Why object abstraction?

Objects appear when Python evaluates expressions

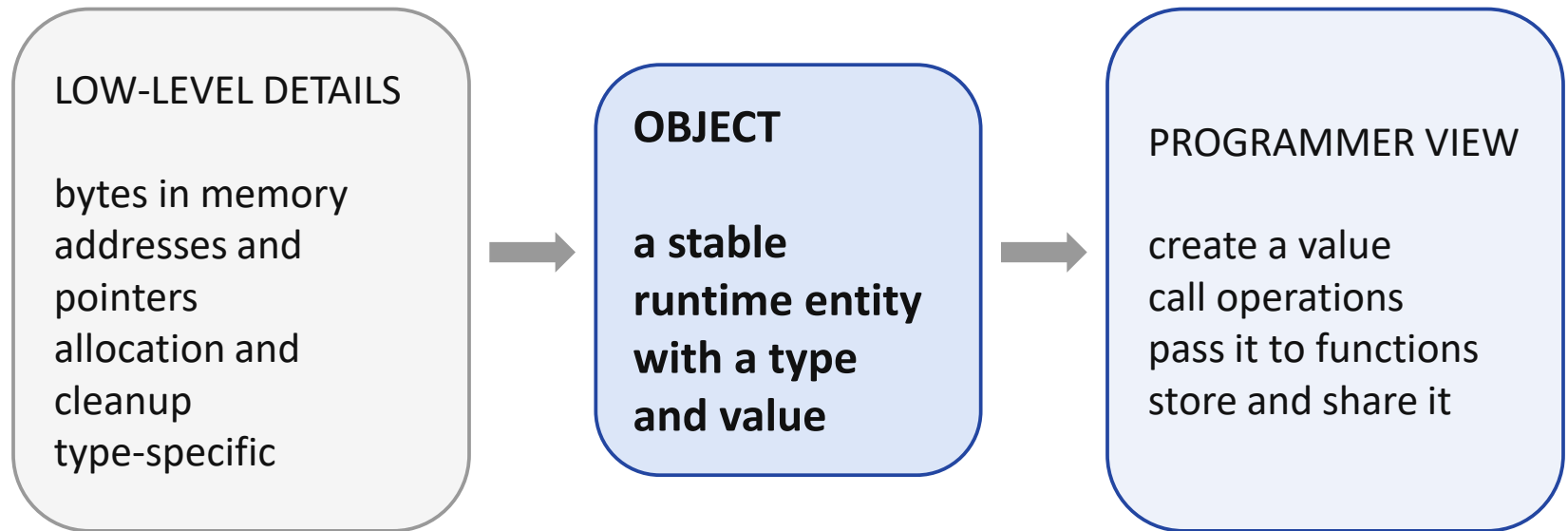


Source code describes actions. Objects exist at runtime.

This is the boundary between language syntax and runtime state.

Why object abstraction?

Organizes related data and behavior.



The abstraction hides representation details without making them disappear.

Why object abstraction?

Organizes related data and behavior.

Imagine we implement a program to work with scores of the student.
For each student we have hw_grade, exam_grade, project_grade and name.
We want to calculate final score according to grade importance.

```
hw_grade = [8, 10, 7]
exam_grade = [9, 6, 8]
project_grade = [10, 9, 9]
names = ['Alice', 'Bob', 'Charlie']
imp = [0.6, 0.2, 0.2] #grade_importance

final_grades = {}
for h, e, p, n in zip(hw_grade, exam_grade, project_grade, names):
    final_grades[n] = imp[0] * h + imp[1] * e + imp[2] * p
```

For implementation it's not very convenient, since data related to one student are stored in separate lists, not connected to each other.

You need to remember their names, etc. Also, it's not very readable.

If you want to see what information we have for one student – it might be pretty tricky.

Why object abstraction?

Organizes related data and behavior.

```
class Student:
    importance = {'hw': 0.6, 'exam': 0.2, 'project': 0.2}
    def __init__(self, name, hw_grade, exam_grade, project_grade):
        self.name = name
        self.hw_grade = hw_grade
        self.exam_grade = exam_grade
        self.project_grade = project_grade

    def final_grade(self):
        return (self.hw_grade * self.importance['hw']
                + self.exam_grade * self.importance['exam']
                + self.project_grade * self.importance['project'])
```

Once you have implemented that, you may forget how final grade is calculated.
This logic is now hidden.

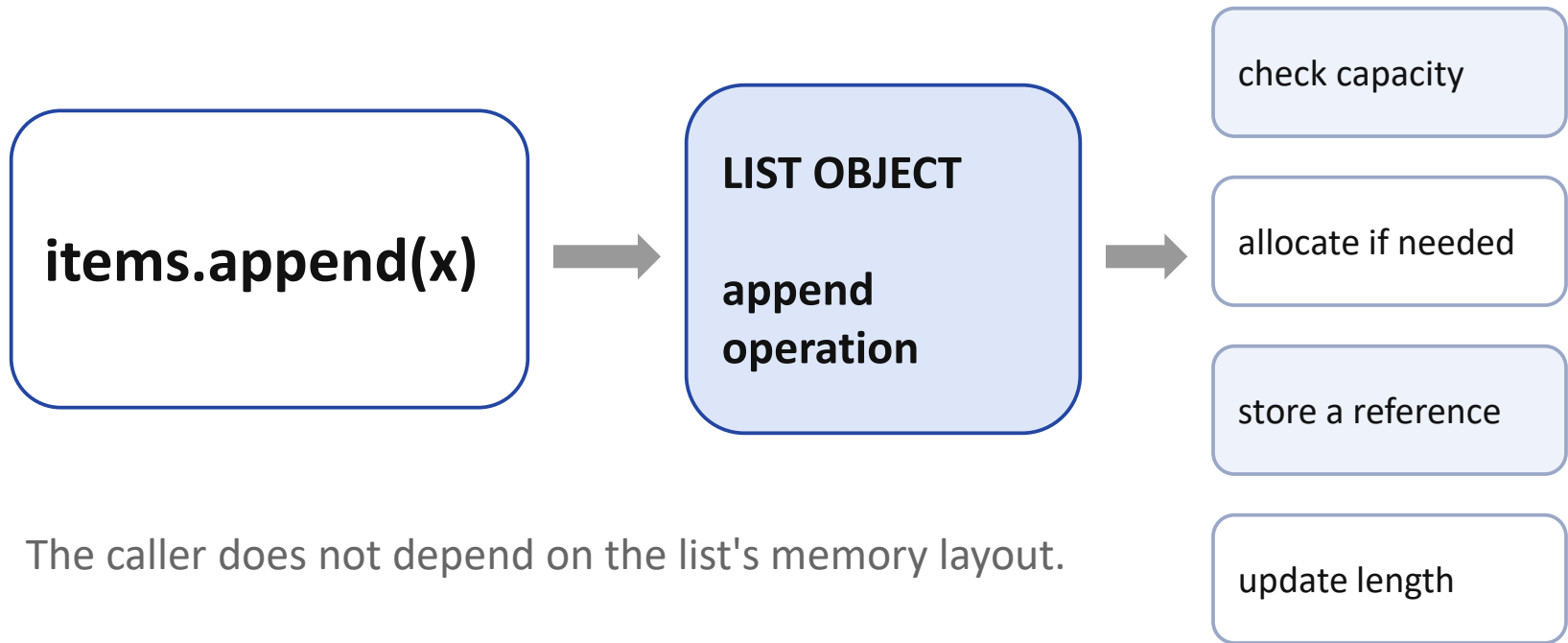
```
students = [Student('Alice', 8, 9, 10),
             Student('Bob', 10, 6, 9),
             Student('Charlie', 7, 8, 9)]
final_grades = {s.name: s.final_grade() for s in students}
```

Why object abstraction?

Objects localize complexity.

Code uses a stable operation while the runtime handles implementation details

The same with other cases. You don't need to know how `list.append` works, you just use it. This leads to term *encapsulation*; we will discuss later.



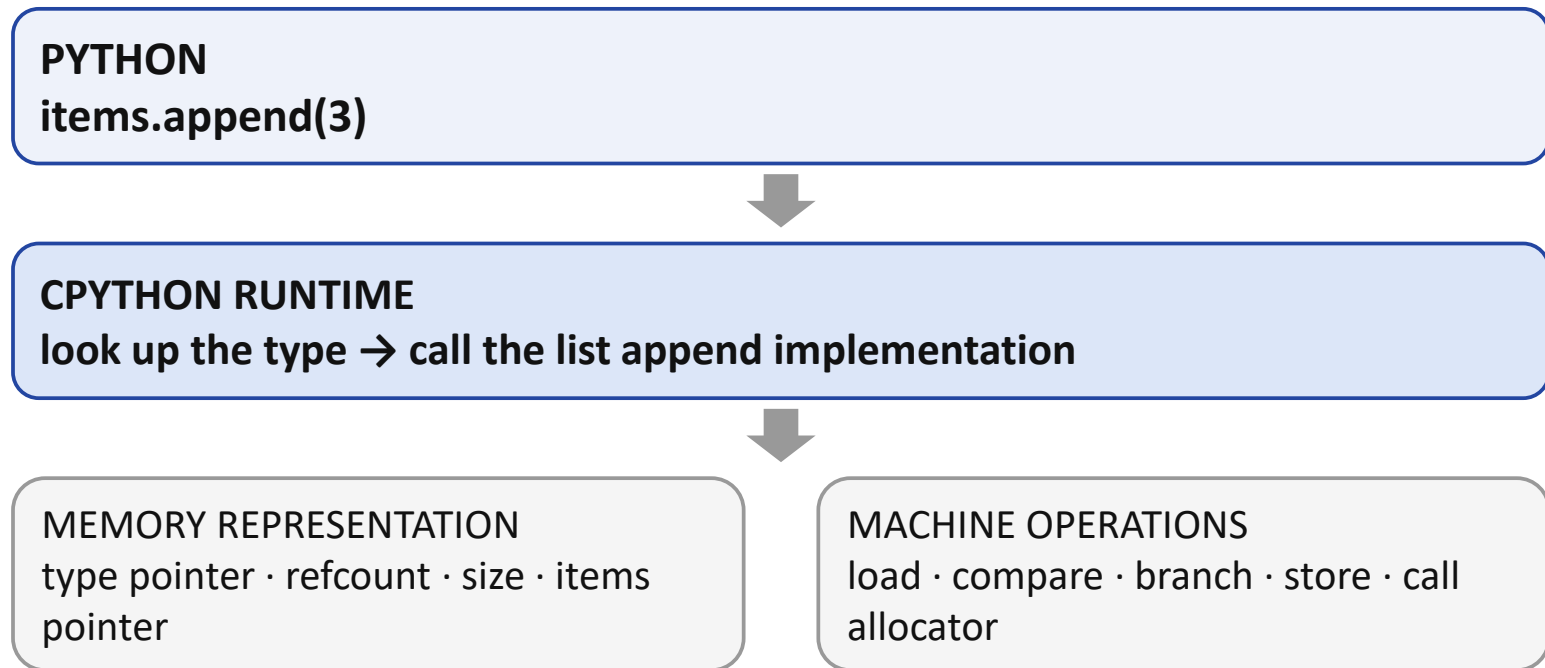
The caller does not depend on the list's memory layout.

Internals can change while the code that uses the object stays the same.

Why object abstraction?

Objects are not a CPU feature

The runtime translates object operations into memory accesses and instructions

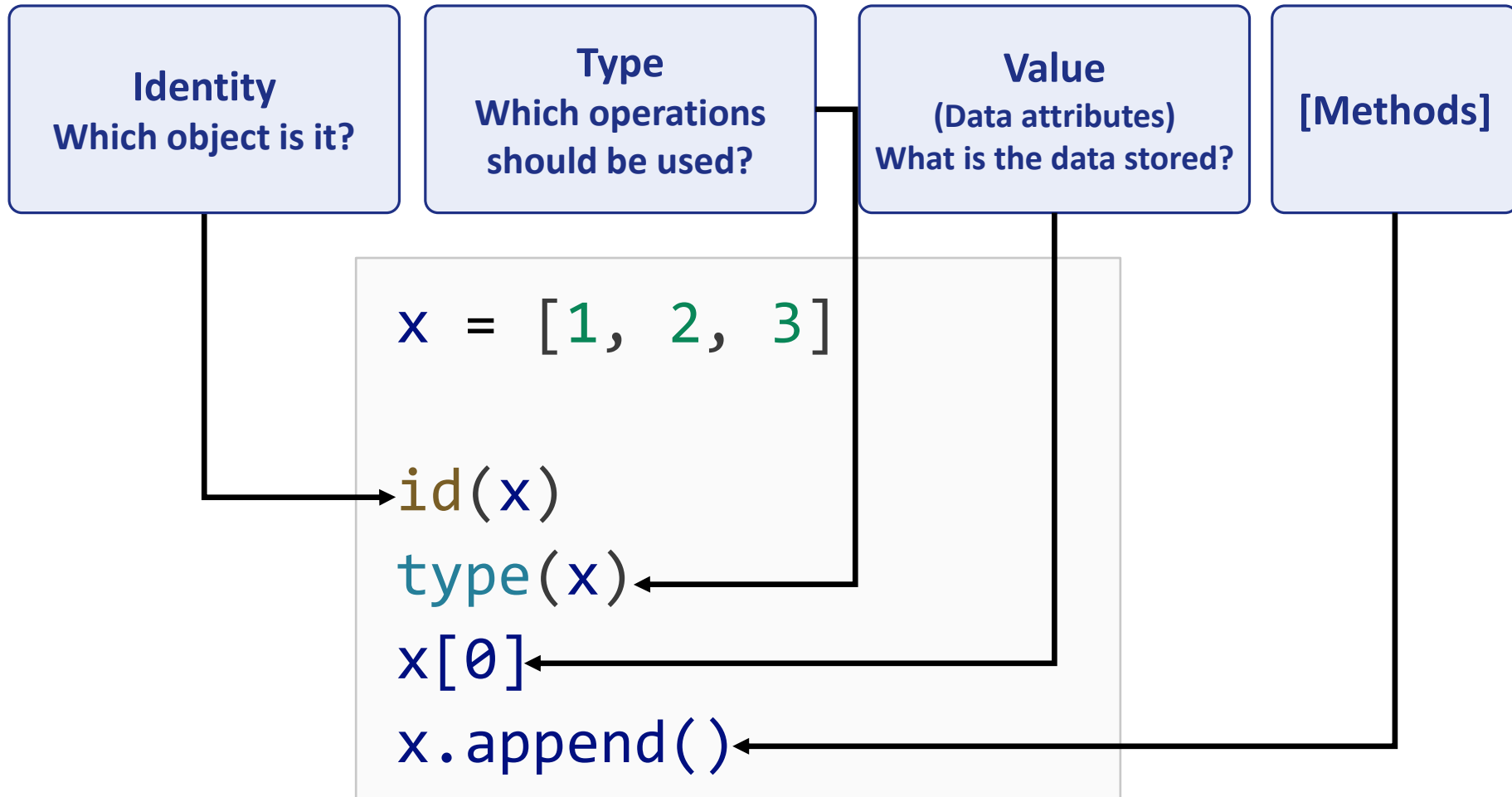


The CPU sees addresses, bytes, and instructions. “Object” is the runtime's organizing abstraction.

Why object abstraction?

What is an object?

A runtime entity with identity, type, and value



Its type determines which operations the object supports.

Why object abstraction?

Different values, one object model

Very different values follow the same runtime rules

```
x = 42  
s = "hello"  
a = [1, 2, 3]  
f = len
```

42
int object

"hello"
str object

[1, 2, 3]
list object

len
builtin function

“Everything is an object”: central Python design principle.

Why object abstraction?

Different values, one object model

Same VM instructions executed differently according to type

```
def aplusb(a, b):  
    return a + b
```

```
1  RESUME          0  
2  LOAD_FAST_BORROW 1 (a, b)  
   BINARY_OP       0 (+)  
   RETURN_VALUE
```

```
apusb(10, 20)           # -> 30  
apusb('abc', 'de')      # -> abcde  
apusb([1, 2], [3, 4])    # -> [1, 2, 3, 4]
```

Runtime decides which implementation to use depending on the type.
That provides python flexibility and dynamic typing.

“Everything is an object”: central Python design principle.

Why object abstraction?

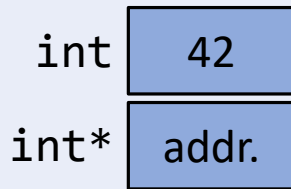
Names and references

Names and references

Stack vs Heap

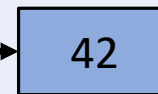
Stack

Function calls / stack frames
Local variables, parameters, return info
Automatic lifetime: created on call,
removed on return
Usually fast and structured, but limited
in size



Heap

Dynamically allocated memory / objects
Flexible lifetime: can outlive a function call
Used for data structures, objects, dynamic
storage
Usually larger, but access/management is
more complex



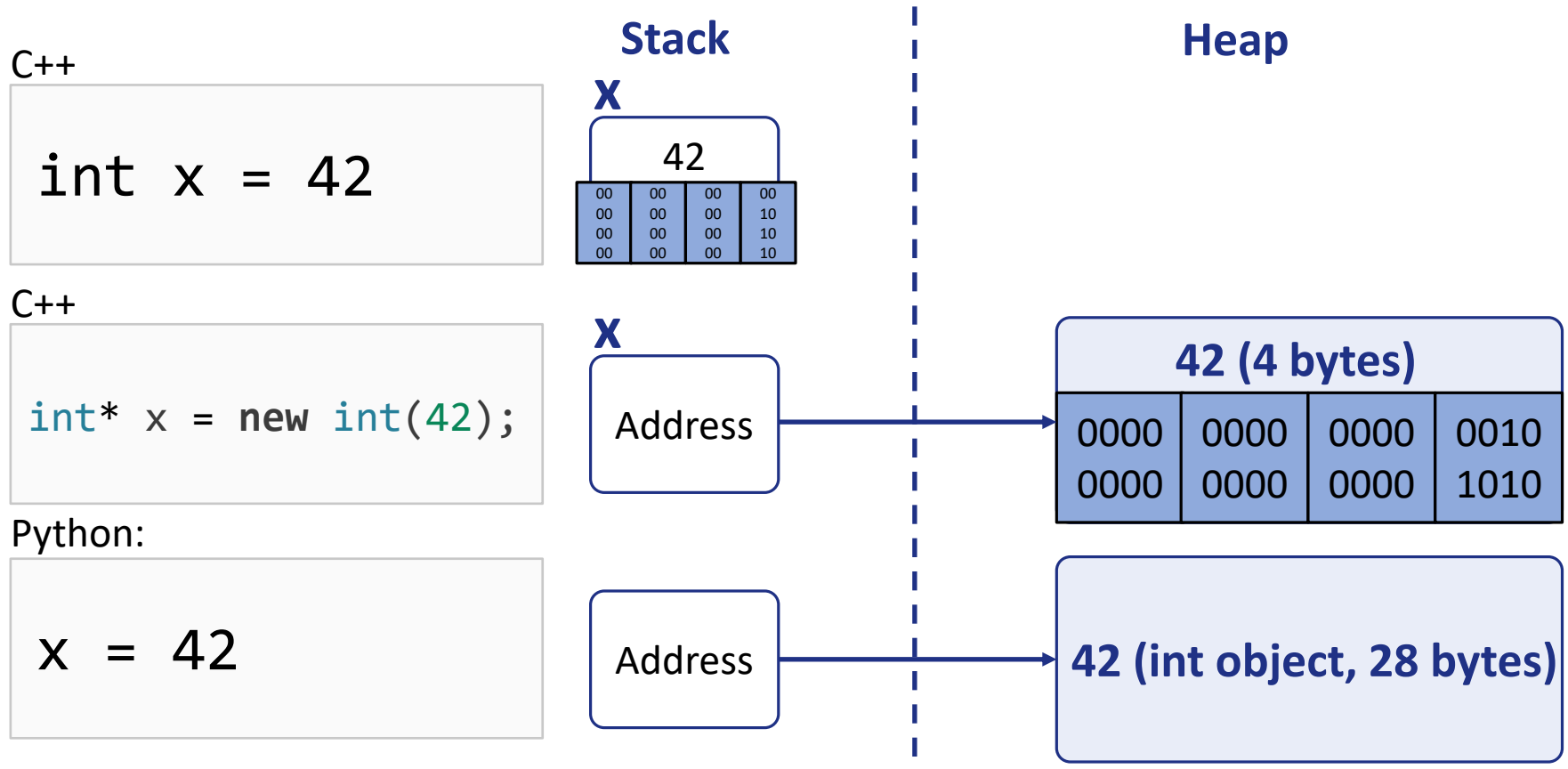
C++

```
int x = 42;  
// stack frame  
// deleted automatically  
// when function is complete
```

```
int* p = new int(42);  
// int object allocated  
// dynamically  
// Deleted manually  
// When needed:  
delete p;
```

Names and references

A Python variable is a reference (pointer)



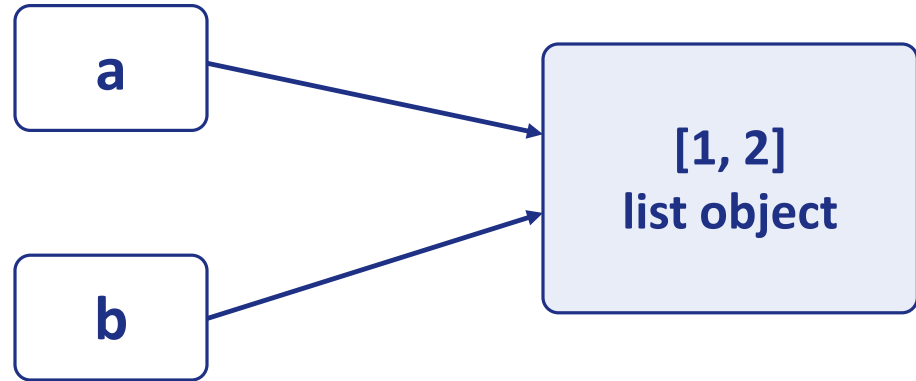
Assignment binds a name to an object.

It does not “put a raw integer into `x`”.

Names and references

Two names may refer to the same object

```
a = [1, 2]  
b = a
```



b = a does not copy the list.

Names and references

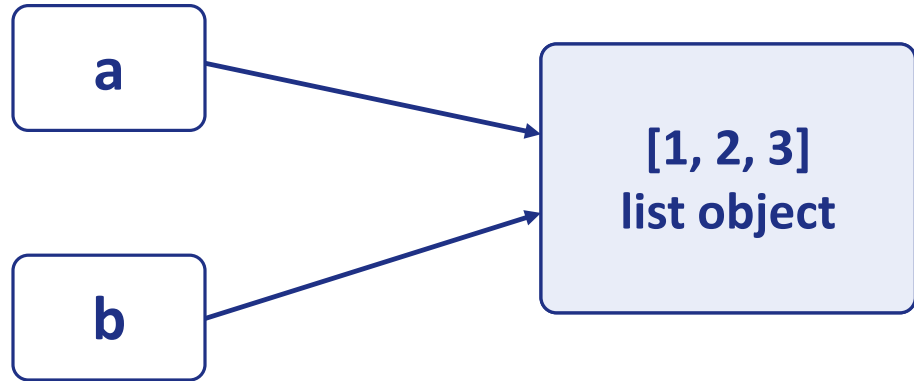
Mutation is visible through every reference

```
a = [1, 2]
```

```
b = a
```

```
b.append(3)
```

```
print(a)
```

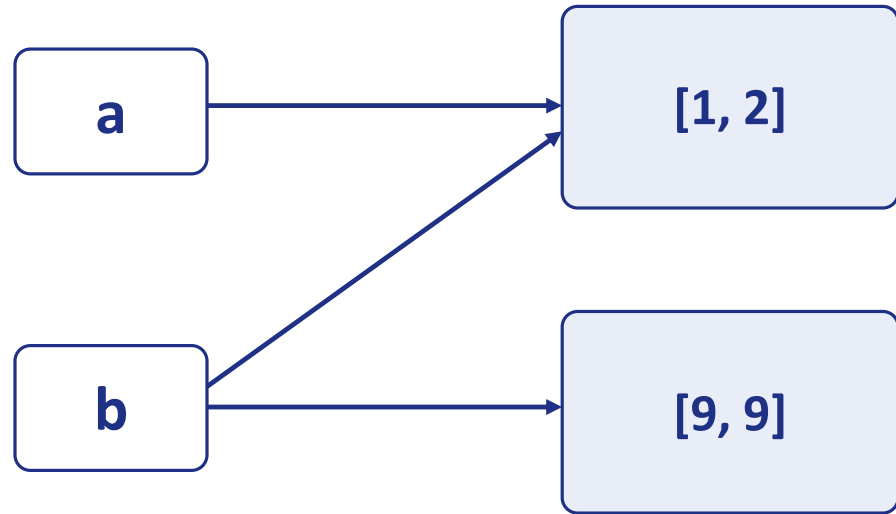


The list changed. The names did not.

Names and references

Rebinding a name does not mutate the old object

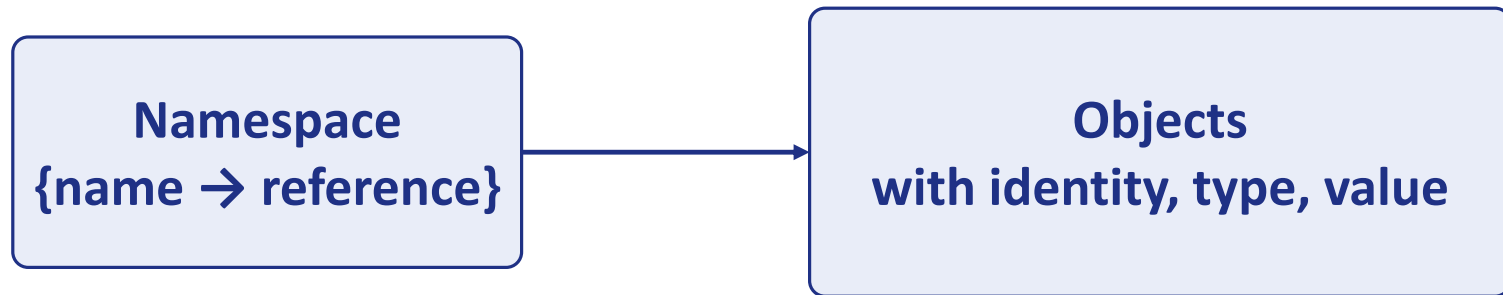
```
a = [1, 2]  
b = a  
b = [9, 9]
```



Assignment changes a binding. Mutation changes an object.

Names and references

A useful mental model



Names live in namespaces. Objects exist independently of any particular name.

This model explains many “weird” Python behaviors.

Names and references

A useful mental model

Mutability

Can an object change after creation?

Mutability

Can an object change after creation?

Two fundamentally different behaviors

IMMUTABLE

int · float · bool

str

tuple*

frozenset

MUTABLE

list

dict

set

most user-defined objects

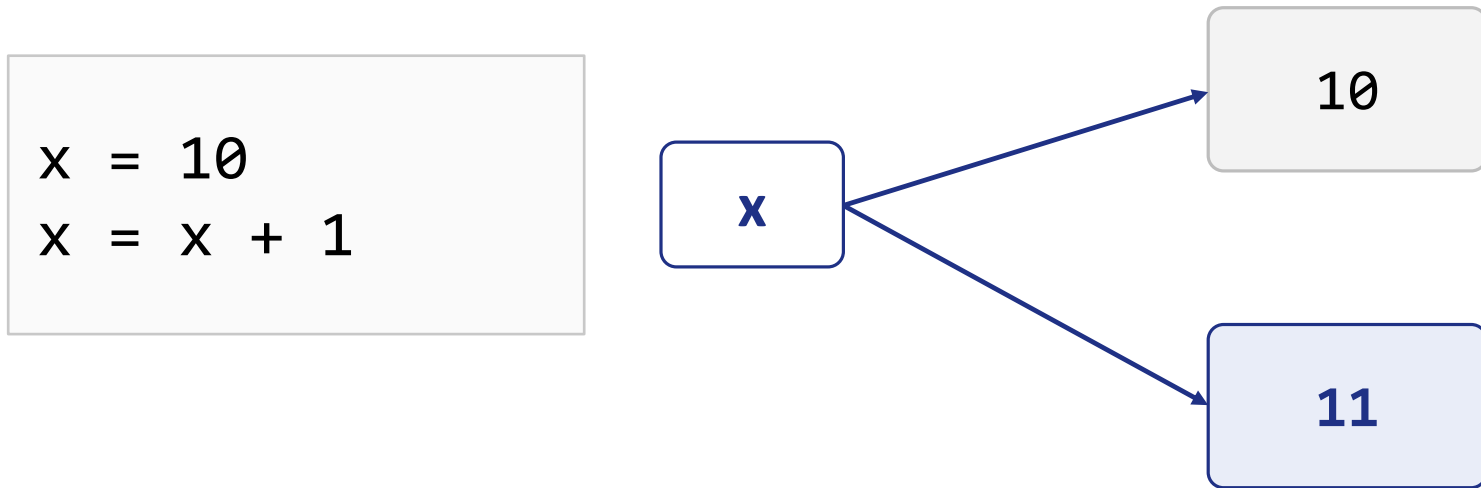
* A tuple is immutable, but it may reference mutable objects.

Immutable: the object's value cannot change in place.

Mutability

Can an object change after creation?

Integers do not change in place



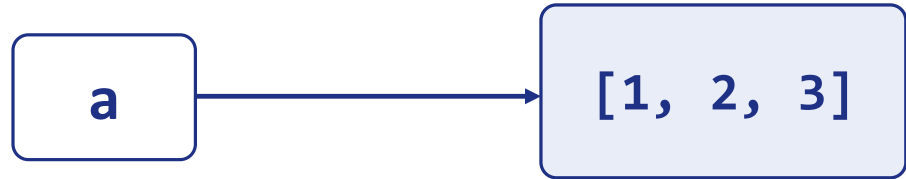
`x = x + 1` creates / obtains another integer object and rebinds `x`.

Mutability

Can an object change after creation?

Lists can change in place

```
a = [1, 2]  
a.append(3)
```



Mutation keeps the binding and changes the object.

Mutability

Can an object change after creation?

Mutation and rebinding are different operations

OBJECT CHANGES

```
a.append(3)
```

```
a[0] = 100
```

the existing list is mutated

BINDING CHANGES

```
a = [9, 9]
```

```
a = a + [3]
```

a starts referring elsewhere

Ask: did the object change, or did the name start referring elsewhere?

Mutability

Can an object change after creation?

The same syntax may mean mutation or rebinding

```
a = [1, 2]  
b = a  
a += [3]
```

**list: usually mutates in
place**

```
x = 10  
y = x  
x += 1
```

int: creates/rebinds value

Never reason from syntax alone — reason from the object type and operation.

Mutability

Can an object change after creation?

Identity vs equality

Identity vs equality

Two different questions

a == b

Do the objects have equal values?

a is b

Are these references to
the very same object?

Equality is about values. Identity is about object identity.

Identity vs equality

Equal lists can still be different objects

a is b

$\Rightarrow$
 $\nLeftarrow$

a == b

```
a = [1, 2]
```

```
b = [1, 2]
```

```
a == b    # True
```

```
a is b    # False
```

a

[1, 2]
object A

b

[1, 2]
object B

Same contents, different identities.

Identity vs equality

The important everyday use of is

```
if result is None:  
    ...
```

None

None is a singleton: code conventionally checks its identity.

Use == for values. Use is for identity (especially is None).

Identity vs equality

Do not rely on object caching / interning

```
a = 1000
b = 1000

a == b    # meaningful
a is b    # do not rely
```

Implementations may reuse some immutable objects. Those optimizations are not the semantics you should program against.

Identity optimizations are implementation details.

Identity vs equality

Functions and references

Function arguments

Python passes object references by assignment



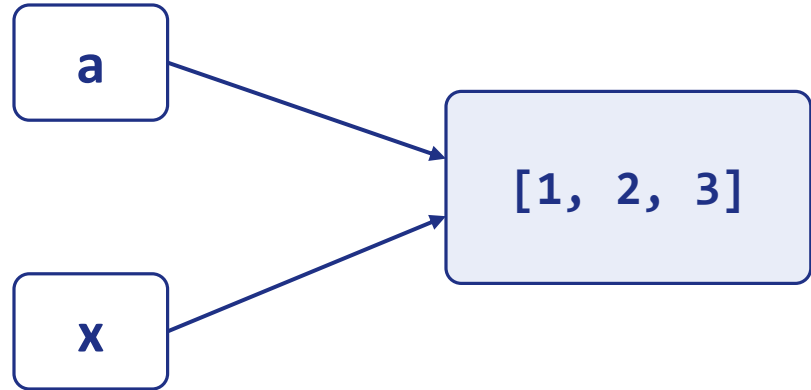
The call creates another local name referring to the same object.

Function arguments

Mutating an argument can affect the caller

```
def add_item(x):  
    x.append(3)
```

```
a = [1, 2]  
add_item(a)  
print(a)
```



Both names still refer to the same mutated list.

Function arguments

Rebinding the parameter does not rebind the caller

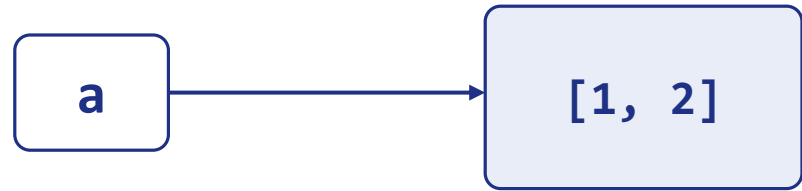
```
def replace(x):  
    x = [9, 9]  
  
a = [1, 2]  
replace(a)  
print(a)
```

Function arguments

Rebinding the parameter does not rebind the caller

```
def replace(x):  
    x = [9, 9]
```

```
a = [1, 2]  
replace(a)  
print(a)
```

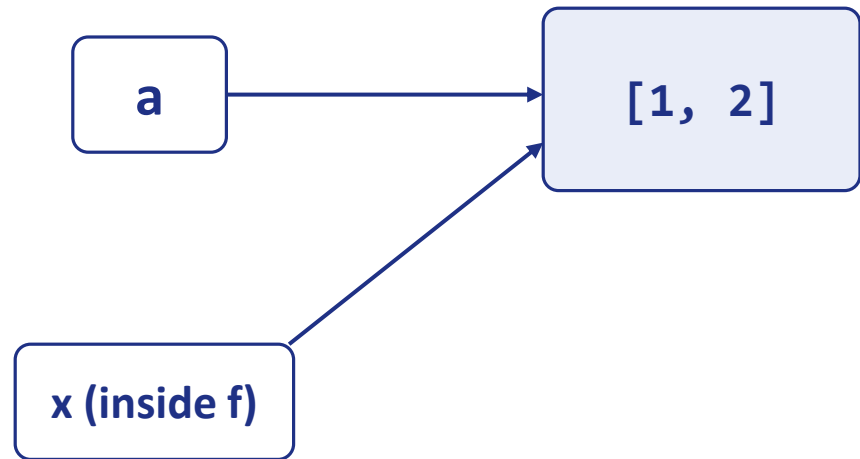


Function arguments

Rebinding the parameter does not rebind the caller

```
def replace(x):  
    x = [9, 9]
```

```
a = [1, 2]  
replace(a)  
print(a)
```

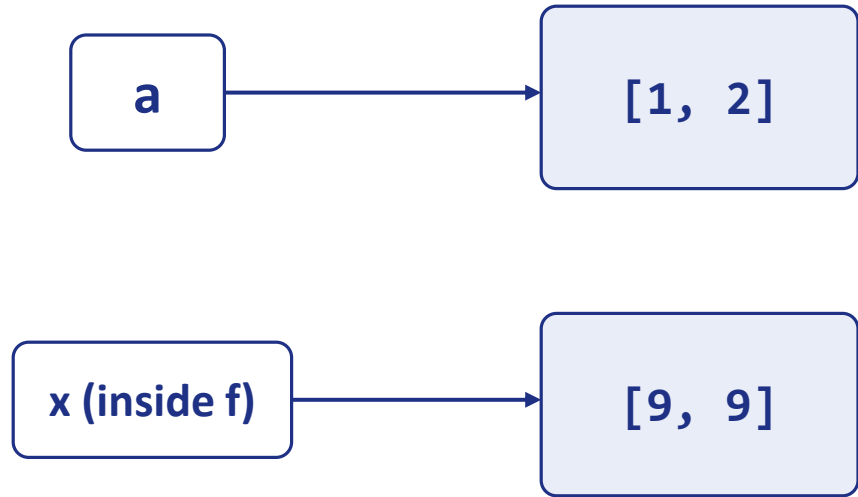


Function arguments

Rebinding the parameter does not rebind the caller

```
def replace(x):  
    x = [9, 9]
```

```
a = [1, 2]  
replace(a)  
print(a)
```

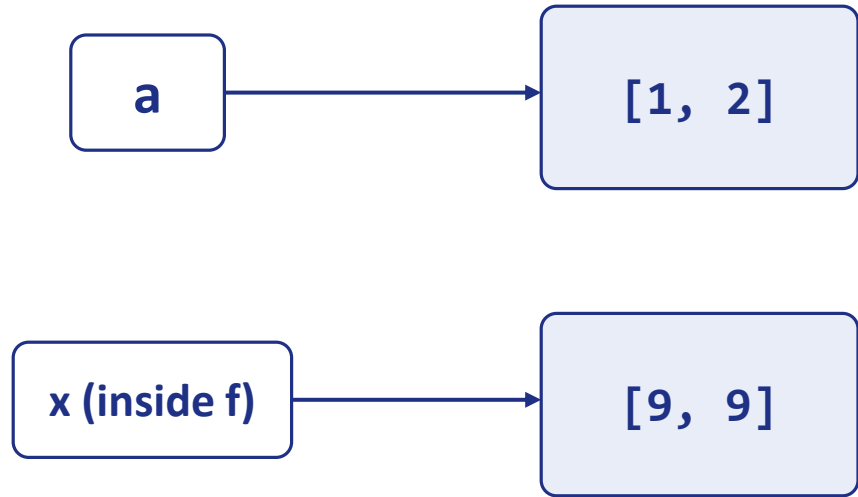


Function arguments

Rebinding the parameter does not rebind the caller

```
def replace(x):  
    x = [9, 9]
```

```
a = [1, 2]  
replace(a)  
print(a)
```

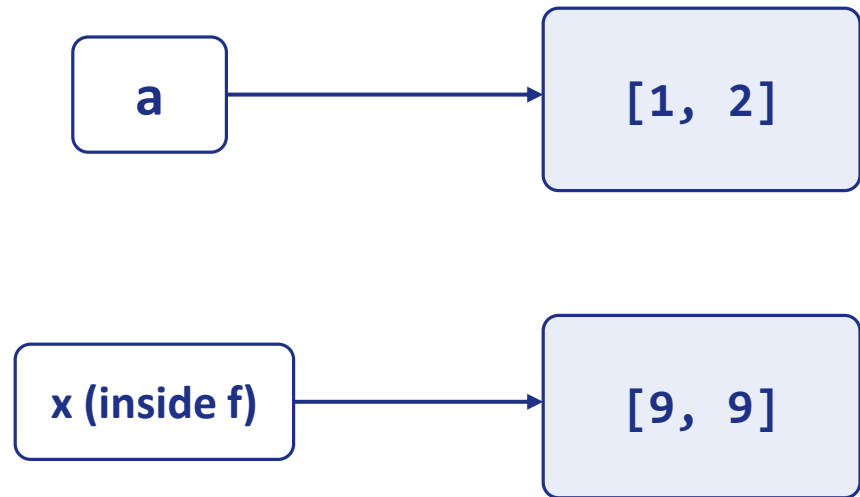


Function arguments

Rebinding the parameter does not rebind the caller

```
def replace(x):  
    x = [9, 9]
```

```
a = [1, 2]  
replace(a)  
print(a)
```



Parameter rebinding is local to the function frame.

Function arguments

Terminology: call by object sharing

- The object itself is not copied by a normal function call.
- A new local parameter name is bound to the same object.
- Mutation may therefore be visible to the caller.
- Rebinding the parameter is local to the callee.

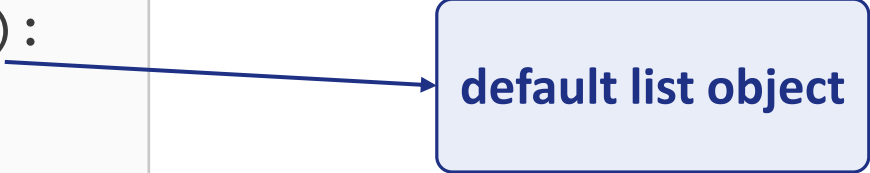
“Call by object sharing” is often the clearest mental model for Python.

Function arguments

Classic pitfall: mutable default arguments

```
def add(x, items=[]):  
    items.append(x)  
    return items
```

```
add(1)    # [1]  
add(2)    # [1, 2] !
```



default list object

The default expression is evaluated when the function is defined, not on every call.

The same list survives across calls.

Function arguments

Common safe pattern

```
def add(x, items=None):  
    if items is None:  
        items = []  
    items.append(x)  
    return items
```

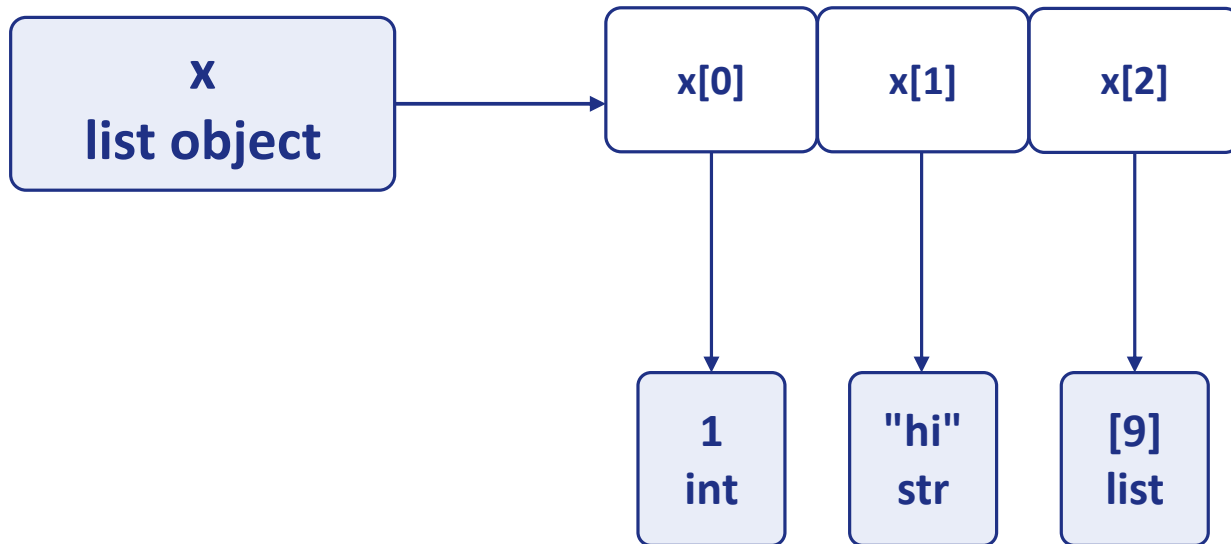
**None is
immutable
and singleton**

Create a fresh mutable object inside the call when needed.

Containers hold references

Containers hold references

A list stores references to objects

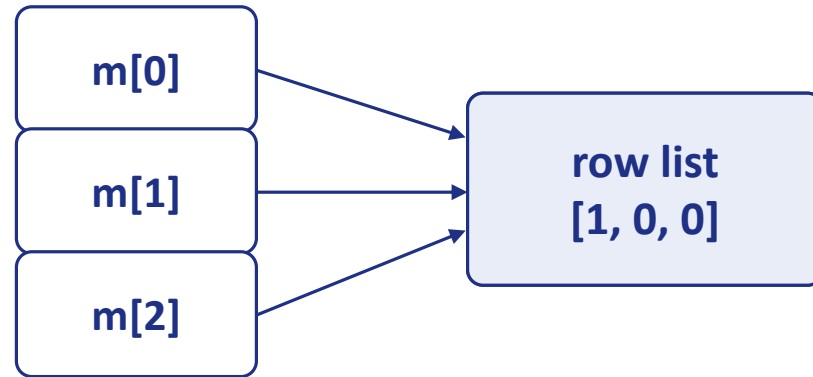


This matters whenever nested mutable objects are shared.

Containers hold references

A famous aliasing trap

```
m = [[0] * 3] * 3  
m[0][0] = 1  
  
print(m)
```



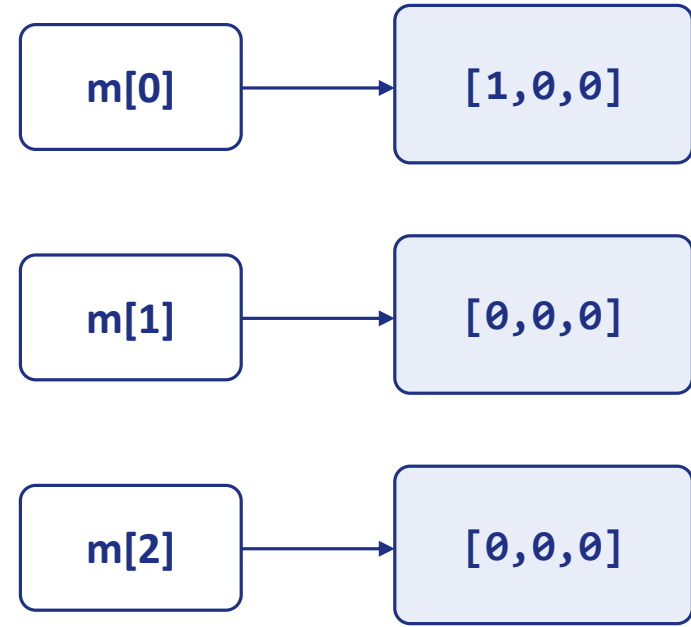
```
[[1,0,0], [1,0,0], [1,0,0]]
```

Three references to one row — not three independent rows.

Containers hold references

Create independent nested objects explicitly

```
m = [[0] * 3 for _ in range(3)]  
m[0][0] = 1
```

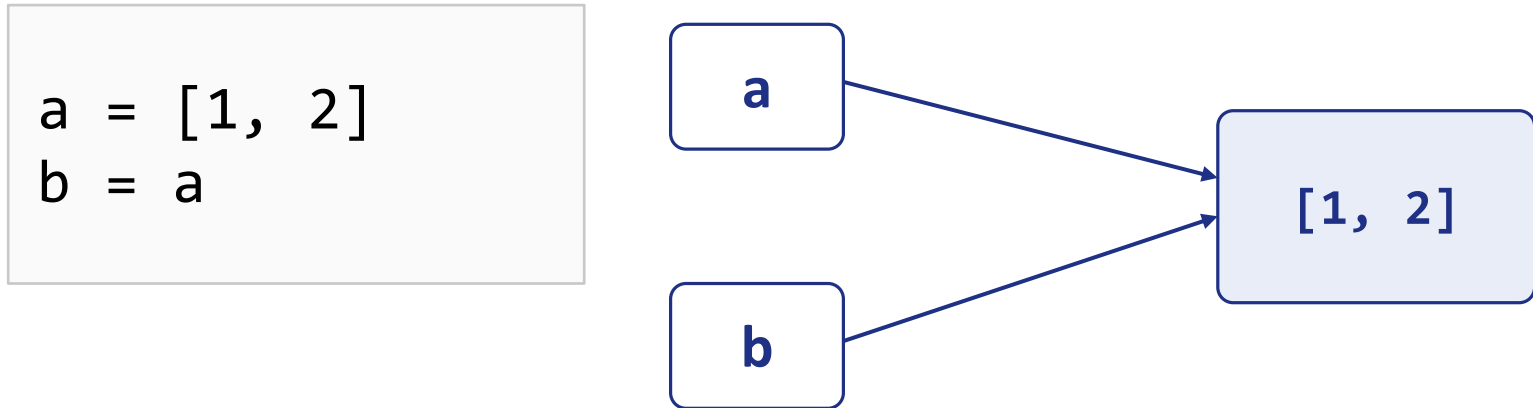


The comprehension evaluates the inner list expression three times.

Copying objects

Copying objects

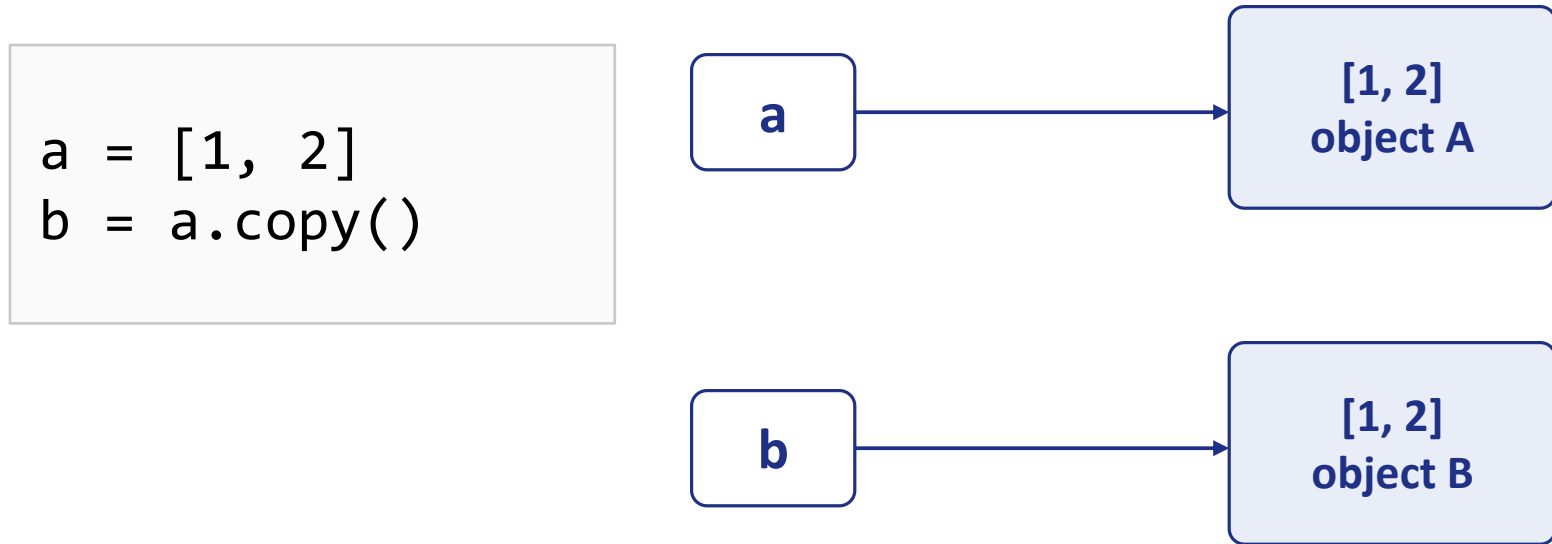
Assignment is not copying



One object, two references.

Copying objects

Shallow copy creates a new outer container

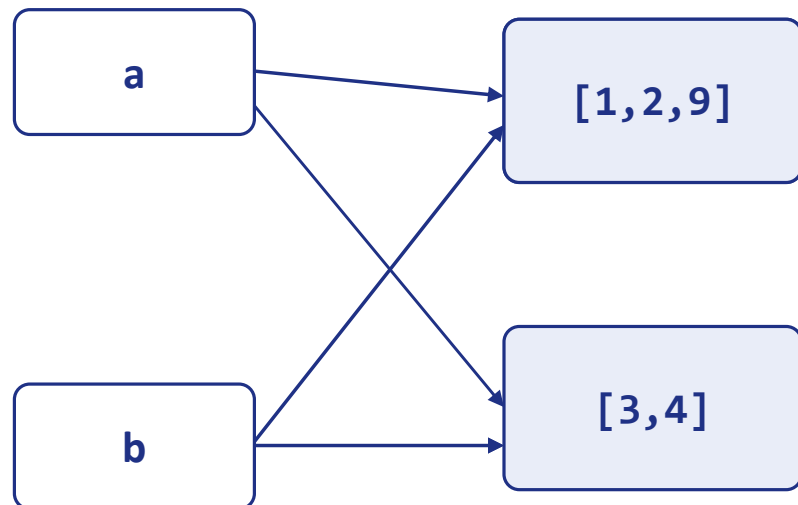


For flat immutable contents, this often looks like an independent copy.

Copying objects

But shallow copy shares nested objects

```
a = [[1, 2], [3, 4]]  
b = a.copy()  
b[0].append(9)
```

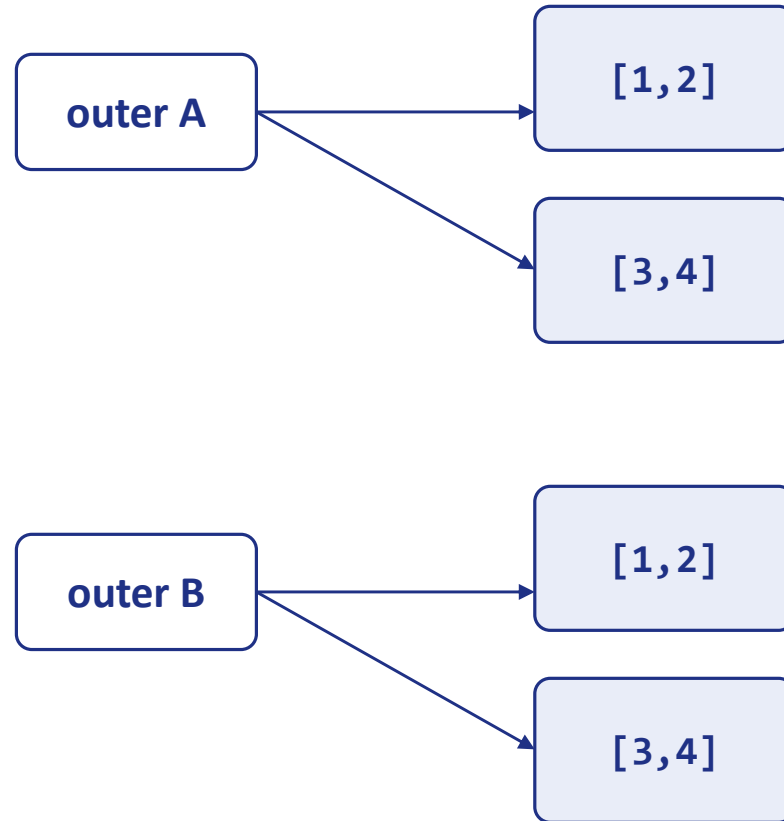


New outer list; same inner lists.

Copying objects

Deep copy recursively duplicates reachable mutable structure

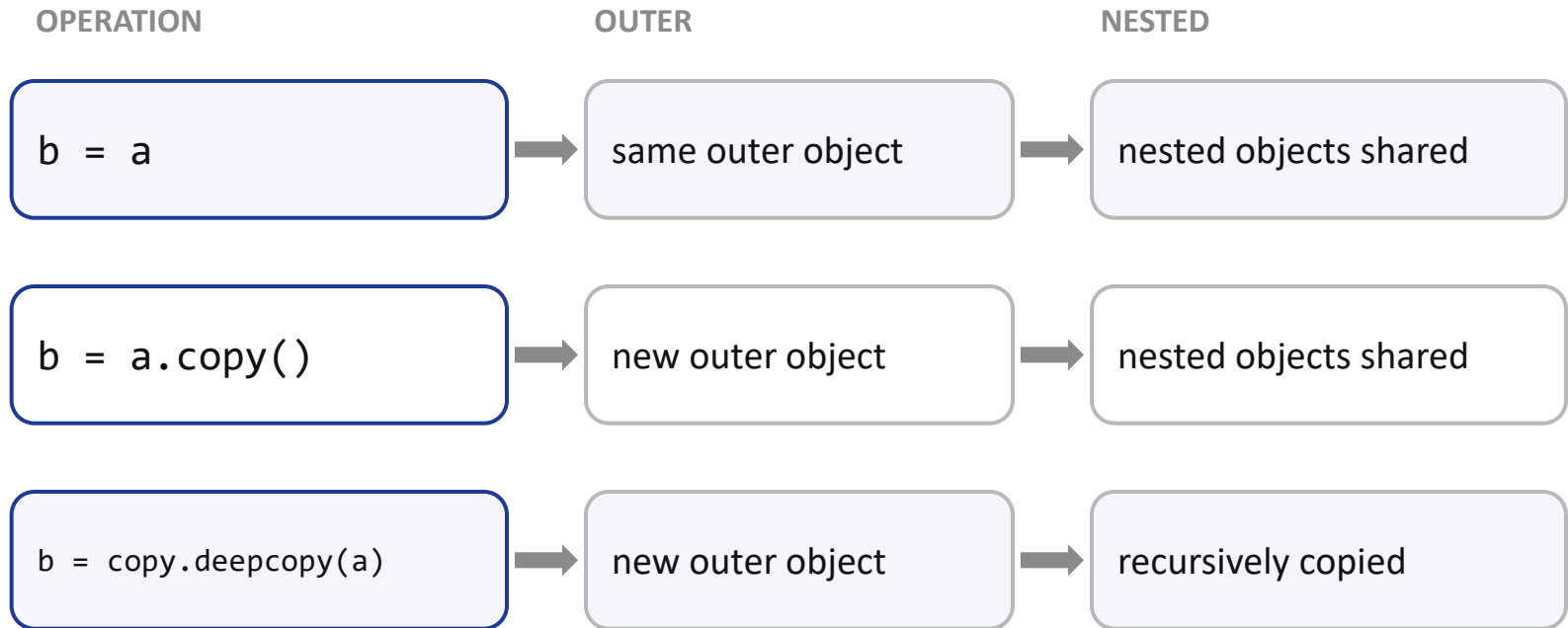
```
import copy  
  
b = copy.deepcopy(a)
```



Deep copy is powerful — and often more expensive than you actually need.

Copying objects

Choose the operation you actually mean



Most copying bugs are aliasing bugs: unexpected sharing of references.

Copying objects

Runtime and memory

Runtime and memory

A function call creates a frame

```
def f(a):  
    x = a + 1 * 2  
    return x
```

f(42)

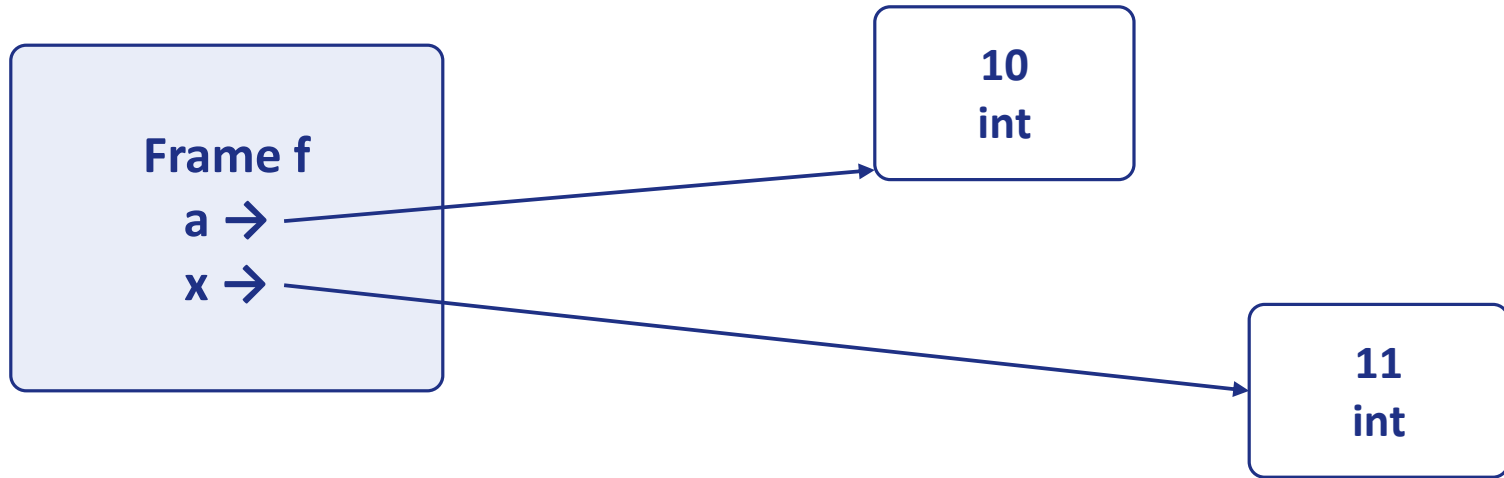
frame for f

local name	reference	Evaluation stack
a	→ object	42
x	→ object	1
		2

A frame contains local bindings and execution state.

Runtime and memory

Frames contain references; objects have separate lifetimes



When the frame disappears, its references disappear. The objects survive only if something else still refers to them.

Runtime and memory

The Python runtime

What does CPython do while your code runs?

Bytecode

execute Python instructions

Frames & calls

locals, parameters, execution state

Object model

types, identity, operations

Memory

allocate, retain, and reclaim

Exceptions

propagate errors through frames

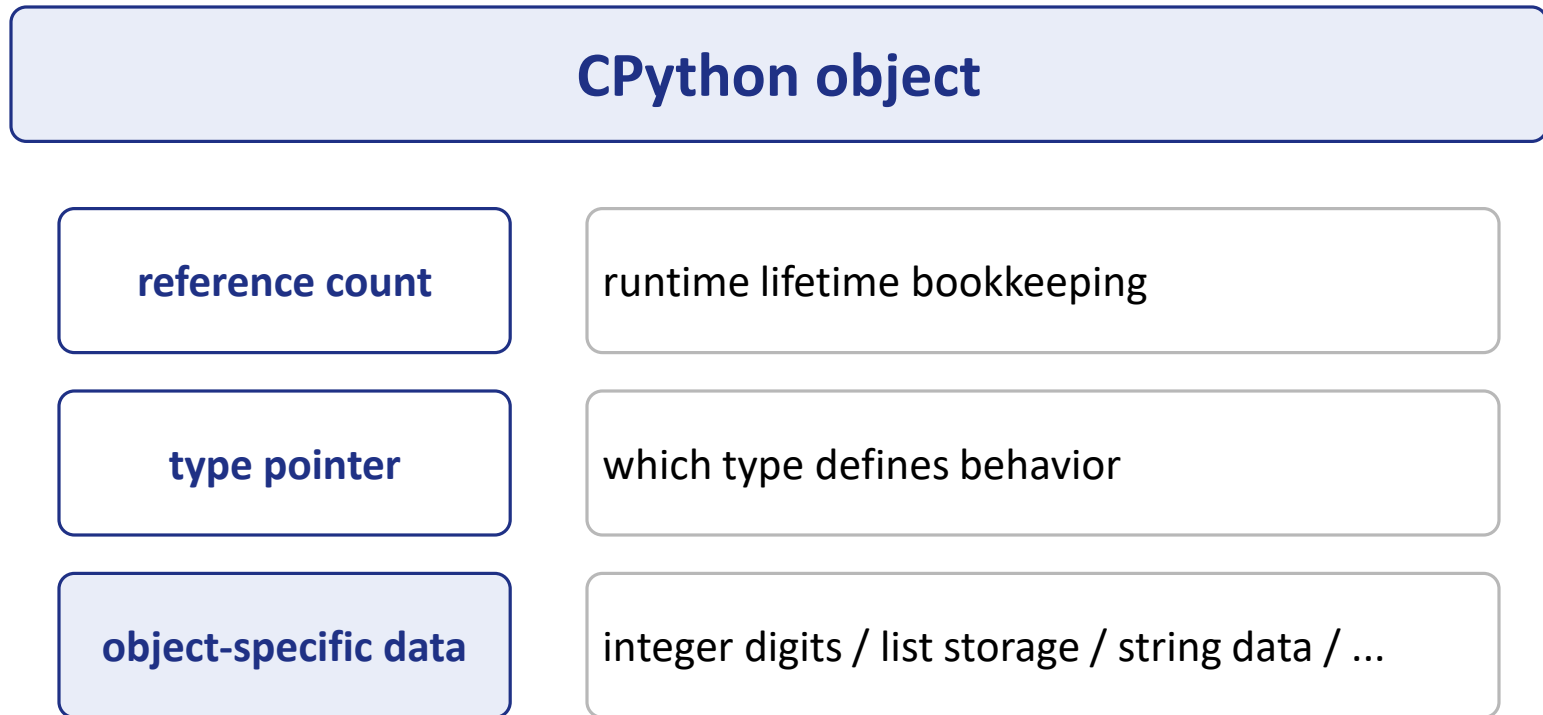
OS interaction

files, sockets, processes,

Each layer hides lower-level machinery while exposing a simpler model above.

Runtime and memory

The CPython objects



Exact layout is an implementation detail; the key idea is metadata + payload.

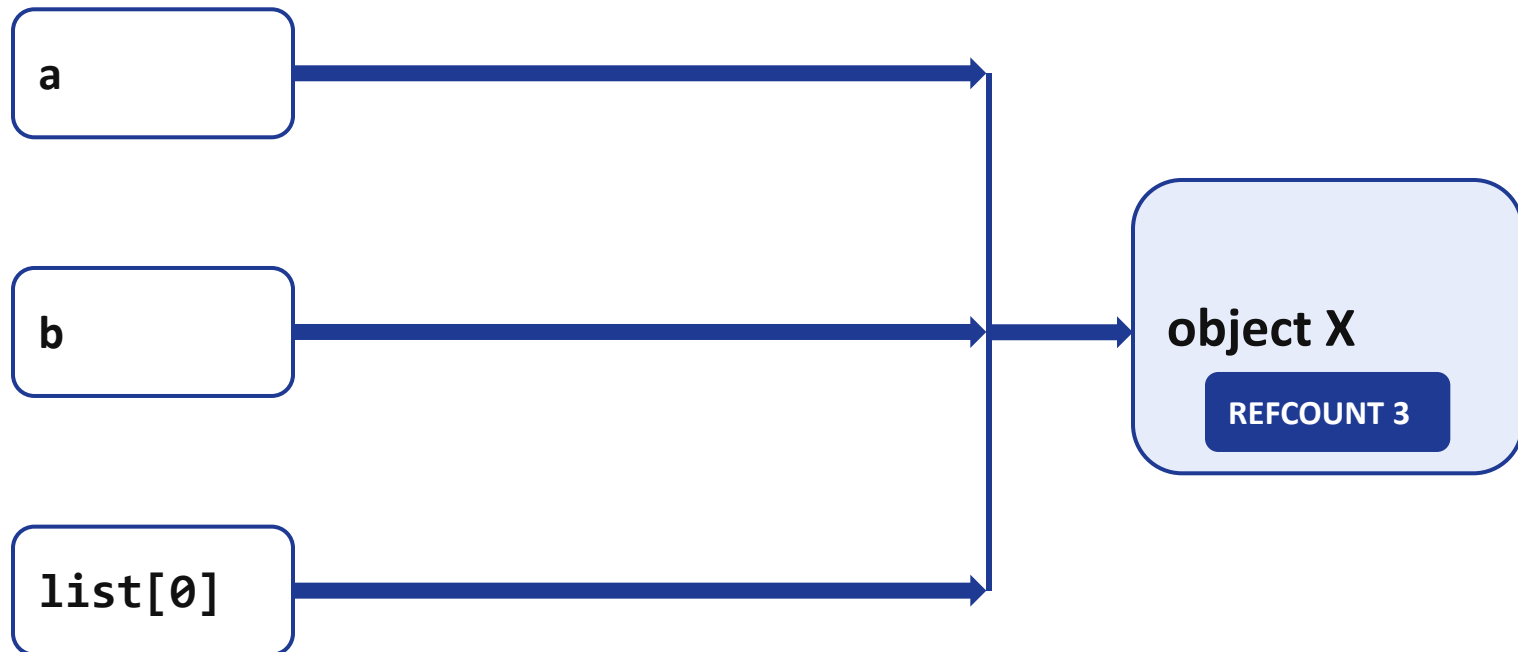
Object lifetime

Reference counting

Object lifetime

Reference counting

Each strong reference contributes to object lifetime

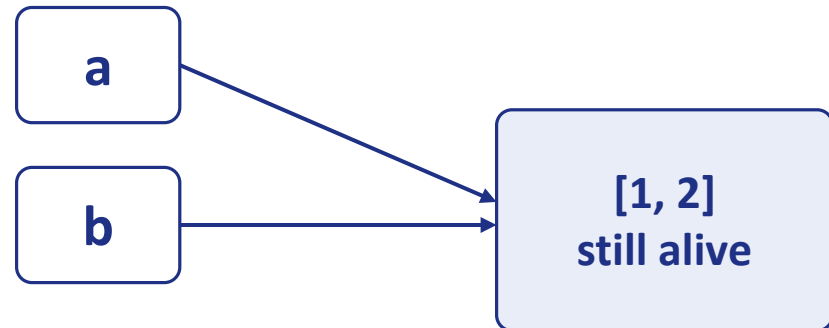


Object lifetime

Reference counting

del removes a binding — not necessarily the object

```
a = [1, 2]  
b = a  
del a
```

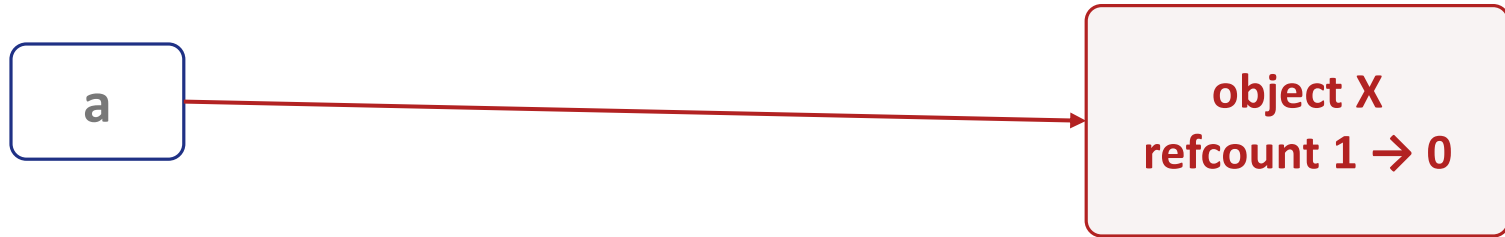


The object is collectible only when no strong references remain (ignoring special runtime details).

Object lifetime

Reference counting

When the last reference disappears:



The reference is removed.

CPython can usually reclaim the object immediately.

Deterministic-looking cleanup is a CPython property, not a universal Python guarantee.

Object lifetime

Cyclic Garbage Collector

Cycles are the classic problem:

```
a = []  
b = []  
a.append(b)  
b.append(a)  
  
del a, b
```



No external names remain, but the objects still refer to each other.

Reference counting alone cannot reclaim every unreachable cycle.

CPython also detects unreachable reference cycles using Cyclic Garbage Collector. If a group of container objects is unreachable from the running program, cyclic GC can reclaim it.

Object lifetime

`id()`, `type()` and `sys.getsizeof()`

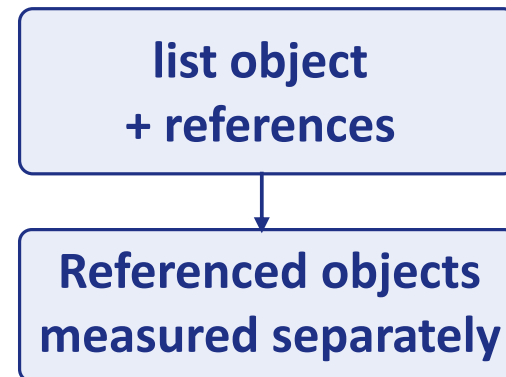
```
import sys

x = [1, 2, 3]
print(type(x))
print(id(x))
print(sys.getsizeof(x))
```

- `id(x)`: identity token unique during the object lifetime
- `type(x)`: the object's type
- `getsizeof(x)`: shallow size reported by this Python implementation

Container size is not the size of everything it references:

```
a = [1]
b = ["a very long string,
pretty long, soooooooo long"]
print(sys.getsizeof(a)) → 64
print(sys.getsizeof(b)) → 64
```



Conclusion

What should you remember?

- Python names are bindings to objects, not bytes containing raw values.
- Objects have identity, type and value.
- Mutable objects can change in place; immutable objects cannot.
- `==` compares values; `is` compares identity.
- Function parameters are new names bound to the same passed objects.
- Containers store references, which creates aliasing and copying pitfalls.
- CPython primarily uses reference counting + cyclic garbage collection to detect objects to remove.

Thank you for watching!